
Faculty of Engineering, Built Environment and Information Technology

Fakulteit Ingenieurswese, Bou-omgewing en Inligtingtegnologie / Lefapha la Boetšenere, Tikologo ya Kago le Theknolotši ya Tshedimošo


Department of Computer Science
Faculty of Engineering, Built Environment & IT
University of Pretoria

# COS212 - Data structures and algorithms

## Assignment 3 Specifications: Graphs

Release date: 11-05-2026 at 06:00

Due date: 27-05-2026 at 23:59

Total marks: 354

# Contents

# 1 General Instructions

- ***Read the entire assignment thoroughly before you start coding.***
- This assignment should be completed individually; no group effort is allowed.
- **To prevent plagiarism, every submission will be inspected with the help of dedicated software.**
- Be ready to upload your assignment well before the deadline. There is a late deadline which is 1 hour after the initial deadline which has a penalty of 20% of your achieved mark. **No extensions will be granted.**
- You may not import any of the built-in Java data structures. Doing so will result in a zero mark. You may only use native 1-dimensional arrays where applicable.
- If your code does not compile, you will be awarded a mark of zero. Only the output of your program will be considered for marks, but your code may be inspected for the presence or absence of certain prescribed features.
- If your code experiences a runtime error, you will be awarded a zero mark. Runtime errors are considered unsafe programming.
- **Ensure your code compiles with Java 8**
- The usage of ChatGPT and other AI-Related software to generate submitted code is strictly forbidden and will be considered as plagiarism.
- The use of this practical, in any form, by any company/business/organisation outside the University of Pretoria is strictly forbidden. This includes, but is not limited to, the use of the practical for discussion sessions, review sessions, marketing tools, providing certain aspects as a free service, or publishing the specification on a website.
- Note that plagiarism is considered a very serious offence. Plagiarism will not be tolerated, and disciplinary action will be taken against offending students. Please refer to the University of Pretoria's plagiarism page at `https://portal.cs.up.ac.za/files/departmental-guide/`.

# 2 Overview

Welcome to the Factory Logistics Network, where every enchanted workstation is a *vertex* and every material transfer between stations is a directed, weighted *edge*. A Miner, Smelter, Constructor, Assembler, Manufacturer, Refinery, or Generator becomes a node in the system, while each shipment of iron, coal, copper, caterium, or aluminum becomes a labeled connection with a numerical "cost" (the edge weight). As production grows, the network expands into a busy map of dependencies: some stations feed many others, some receive inputs from across the floor, and every transfer matters.

The factory runs a four-stage optimization ritual to remove inefficiencies and compress redundant structure. First, no station is allowed to dump resources into Miners, and each station keeps only its single most valuable outgoing transfer (its heaviest edge). Next, repeated connection patterns are merged so duplicates collapse into representative stations, and parallel transfers of the same kind are aggregated into one combined flow. Then, self-loops are erased so stations cannot "ship to themselves," and finally, any workstation that ends up isolated is removed from the active layout. The result is a smaller, clearer network that preserves the strongest and most meaningful material routes.

Once the layout is cleaned, the assignment explores two key analyses that guide decision-making. The factory can create a *minimum spanning structure* using Kruskal's method: potential links are considered in increasing weight order. Even though the factory's workflow is directed, the spanning choice is made on undirected pairs, then the original directions that actually exist are copied into the final plan. In parallel, the network can be split into *strongly connected components* using a depth-first search, revealing tightly interdependent clusters of stations where every node can reach every other. Together, these tools help the factory understand which parts of production are essential, which are redundant, and how best to organize transfers to keep the factory profitable and stable.

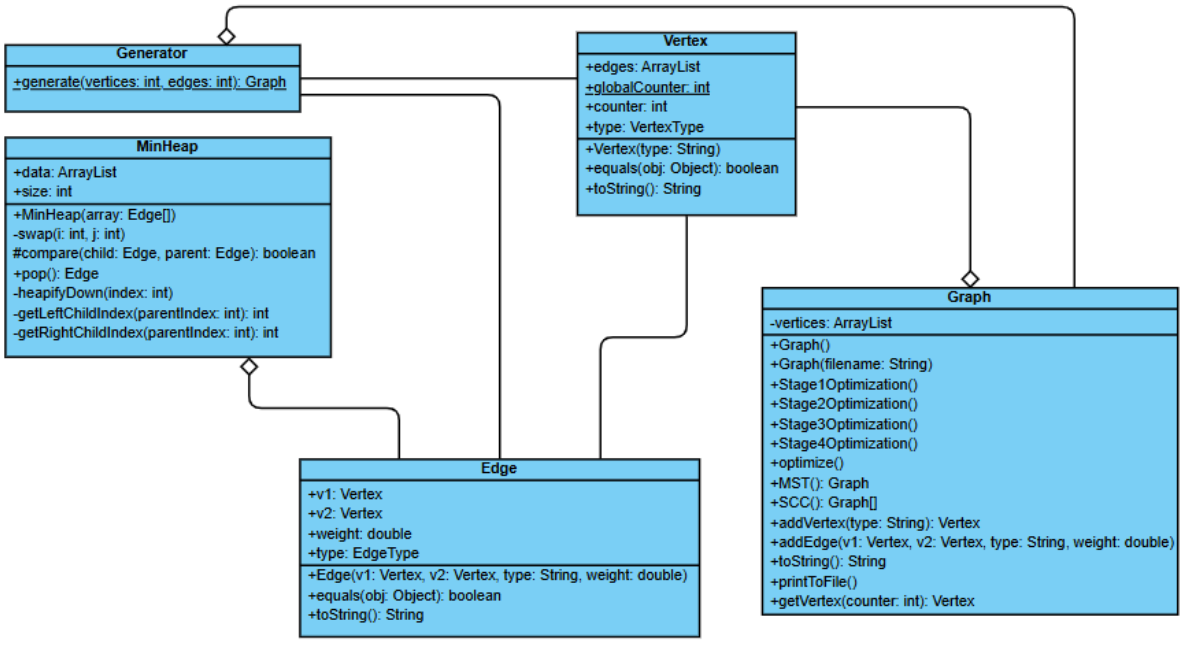


Figure 1: UML Class Diagram

# 3 Generator

This class is a utility used to generate randomised graph structures for testing purposes. It is provided to you to assist in verifying your implementation.

**Do not make any changes to this file, as it will be overwritten on FitchFork.**

## 3.1 Functions

- **+generate(vertices: int, edges: int): Graph**
  - This static method constructs and returns a `Graph` object populated with a random configuration based on the provided parameters.
  - Each edge is assigned a random `EdgeType` and a random `weight` between 1.00 and 10.00, rounded to two decimal places.
  - The resulting vertices and edges are added to the graph using the `addVertex` and `addEdge` methods of the `Graph` class.

# 4 MinHeap

This class implements a Min-Heap data structure specifically designed to store `Edge` objects. It is primarily used to sort edges by weight for Kruskal's algorithm.

**Do not make any changes to this file, as it will be overwritten on FitchFork.**

## 4.1 Members

- **+data: ArrayList**
  - The internal storage for the heap, represented as a dynamic list of edges.
- **+size: int**
  - Tracks the current number of edges within the heap.

## 4.2 Functions

- **+MinHeap(array: Edge[])**
  - The constructor for the `MinHeap`.
  - It populates the `data` list with the provided array and transforms it into a valid min-heap by calling `heapifyDown` on all non-leaf nodes, starting from the bottom up.
- **-swap(i: int, j: int)**
  - A private utility that swaps the positions of two edges in the `data` list.
- **#compare(child: Edge, parent: Edge): boolean**
  - A protected helper method that defines the heap property.
  - Returns `true` if the `child` edge's weight is strictly less than the `parent` edge's weight.
- **+pop(): Edge**
  - Removes and returns the edge with the minimum weight (the root of the heap).

- After removal, the last element is moved to the root position and `heapifyDown` is called to restore the heap property. Returns `null` if the heap is empty.

- **-heapifyDown(index: int)**
  - A private recursive-style utility that maintains the min-heap property by moving an element down the tree until it is smaller than its children or reaches a leaf position.

- **-getLeftChildIndex(parentIndex: int): int**
  - Returns the index of the left child for a given parent using the formula: $2 \times parentIndex + 1$.

- **-getRightChildIndex(parentIndex: int): int**
  - Returns the index of the right child for a given parent using the formula: $2 \times parentIndex + 2$.

# 5 Vertex

This class represents a node within a graph structure, specifically modelled after various industrial machine types. Each vertex is uniquely identified by an auto-incrementing counter and maintains a list of connected edges.

## 5.1 VertexType Enum

This enumeration defines the possible roles a vertex can fulfill:

- **MINER, SMELTER, CONSTRUCTOR, ASSEMBLER, MANUFACTURER, REFINERY, GENERATOR, UNDEFINED**

## 5.2 Members

- **+edges: ArrayList**
  - A list of `Edge` objects that are connected to this specific vertex.

- **+globalCounter: int static**
  - A static tracker used to store the global counter for the vertex id. It starts at 1.

- **+counter: int**
  - The unique identification number for this vertex, assigned from the `globalCounter` upon instantiation.

- **+type: VertexType**
  - Stores the category of the vertex using the `VertexType` enum.

### 5.3 Functions

- **+Vertex(type: String)**
  - The constructor for the `Vertex` class.
  - It initialises the vertex's `counter` and maps the input string (e.g. "miner", "refinery") to the corresponding `VertexType` value. If the string does not match a known type, it defaults to `UNDEFINED`.
- **+equals(obj: Object): boolean**
  - Compares this vertex with another object.
  - Two vertices are considered equal if and only if they share the same `VertexType`.
- **+toString(): String**
  - **Do not change this implementation as it is utilised by FitchFork.**
  - Returns a string representation of the vertex in the format: `(counter image_filename)`.
  - The filename is determined by the vertex type (e.g. a Miner returns `miner.png`).

## 6 Edge

This class represents a connection between two vertices in the graph, representing a resource transport line with a specific resource type and throughput weight.

### 6.1 EdgeType Enum

This enumeration defines the resource types transported by the edge:

- **IRON, COAL, COPPER, CATERIUM, ALUMINUM, UNDEFINED**

### 6.2 Members

- **+v1: Vertex**
  - The source vertex where the edge originates.
- **+v2: Vertex**
  - The destination vertex where the edge terminates.
- **+weight: double**
  - The numerical weight of the edge.
- **+type: EdgeType**
  - Stores the resource category of the edge using the `EdgeType` enum.

## 6.3 Functions

- **+Edge(v1: Vertex, v2: Vertex, type: String, weight: double)**
  - The constructor for the `Edge` class.
  - It assigns the connected vertices and weight, then maps the input string (e.g. "iron", "coal") to the corresponding `EdgeType`. Any unrecognised string defaults to `UNDEFINED`.
- **+equals(obj: Object): boolean**
  - Determines equality between two edges.
  - Returns true if the source (`v1`), destination (`v2`), resource `type`, and `weight` are all identical.
- **+toString(): String**
  - **Do not change this implementation as it is utilised by FitchFork.**
  - Returns a string representation used for system visualisation in the format: `image_filename,weight`.
  - The filename is determined by the resource type (e.g. `iron.png`).

# 7 Graph

This class represents a directed graph structure. It provides functionality to load a graph from a file, export it, and perform various structural optimisations and algorithms like Kruskal's Minimum Spanning Tree and Strongly Connected Components.

## 7.1 Members

- **-vertices: ArrayList**
  - A private list containing all the vertices currently in the graph.

## 7.2 Functions

- **+Graph()**
  - Default constructor that initialises an empty graph with an empty list of vertices.
- **+Graph(filename: String)**
  - Constructor that reads a graph from a file.
  - The file format follows the output of the `toString()` method. It parses vertex IDs, types, and edges to reconstruct the graph state.
- **+Stage1Optimization()**
  - Removes all incoming edges for any vertex of type `MINER`.
  - For every vertex in the graph, it keeps only the single outgoing edge with the highest weight and removes all others. If two edges are equal keep the first edge which is visited.
- **+Stage2Optimization()**
  - Removes duplicate edges connections across the entire graph.
  - Assume the type of H equals the type of A and the type of E equals the type of B And the edge types are the same, then we can replace H–>E with A–>B
  - Imagine you have the following graph:

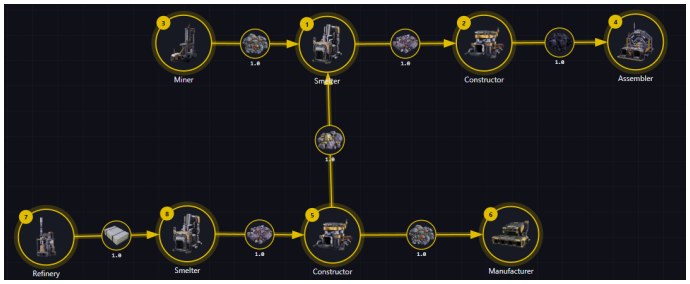


Figure 2: Enter Caption

If 8–>5 == 1–>2

Then Remove 8–>5
And then add the weight from the edge of 8–>5 to the weight of 1–>2

Link 2–>6 and 7–>1
so that it becomes:

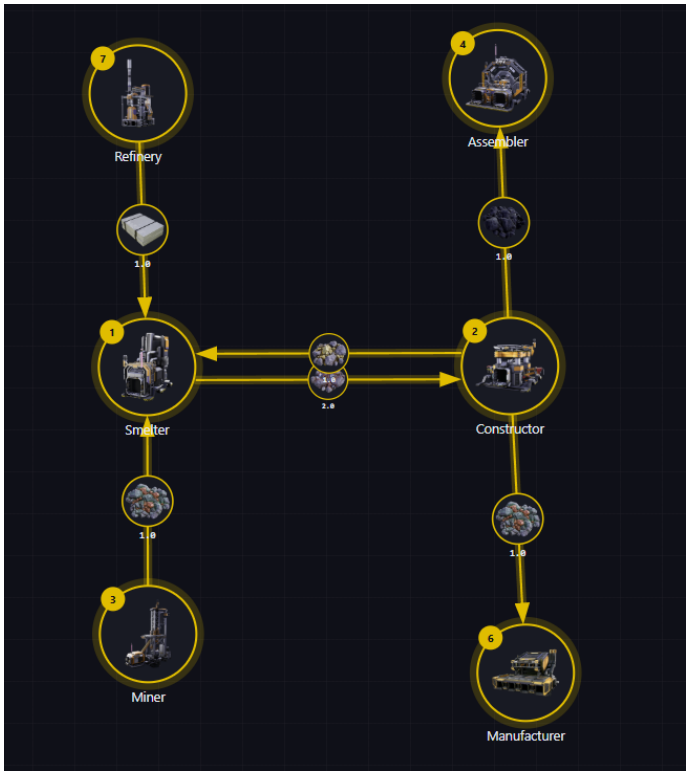


Figure 3: Enter Caption

- **+Stage3Optimization()**
  - Removes all "self-loops" from the graph.
- **+Stage4Optimization()**
  - Removes all isolated vertices from the graph (vertices that have no edges).
- **+optimize()**
  - A template method that executes the four optimisation stages sequentially: Stage 1, then 2, then 3, and finally Stage 4.
- **+MST(): Graph**
  - Generates and returns a new `Graph` representing the Minimum Spanning Tree of the current graph using Kruskal's algorithm.
  - It treats the graph as undirected for this algorithm and utilizes the `MinHeap` for edge sorting.
  - To make the graph undirected duplicate each edge in the opposite direction and after making the MST remove the duplicated edges.
  - If two edges have the same weight then order them based on v1.counter and then v2.counter
- **+SCC(): Graph[]**
  - Returns an array of `Graph` objects, where each graph represents a Strongly Connected Component found in the original graph.

  - SCC's should be returned in the order of the DFS algorithm

- **+addVertex(type: String): Vertex**
  - Creates a new instance of a `Vertex` with the specified type and adds it to the graph's internal vertex list.
  - Returns the newly created `Vertex` object.
- **+addEdge(v1: Vertex, v2: Vertex, type: String, weight: double)**
  - Creates a new `Edge` connecting `v1` (source) to `v2` (destination) with the specified resource type and weight.
  - The edge is then added to the adjacency list (`edges` list) of the source vertex `v1`.
- **+getVertex(counter: int): Vertex**
  - Searches through the graph's vertex list for a vertex with a `counter` (ID) that matches the provided integer.
  - Returns the matching `Vertex` object if found, else it returns `null`.
- **+toString(): String**
  - **Do not change this implementation as it is utilised by FitchFork.**
  - Returns a detailed string representation of the graph, listing all edges and isolated vertices.
- **+printToFile()**
  - **Do not change this implementation as it is utilised by FitchFork.**
  - Exports the current graph's `toString()` output to a file in the `output/` directory with a unique timestamp-based name.

# 8 Testing

As testing is a vital skill that all software developers need to know and be able to perform daily, approximately 10% of the assignment marks will be assigned to your testing skills. To do this, you will need to submit a testing main (inside the Main.java file) that will be used to test an Instructor-provided solution. You may add any helper functions to the Main.java file to aid your testing. In order to determine the coverage of your testing the jacoco tool. The following set of commands will be used to run jacoco:

```
javac *.java
rm -Rf cov
mkdir ./cov
java -javaagent:jacocoagent.jar=excludes=org.jacoco.*,destfile=./cov/output.exec
    -cp ./ Main
mv *.class ./cov
java -jar ./jacococli.jar report ./cov/output.exec --classfiles ./cov --html
    ./cov/report
```

This will generate output which we will use to determine your testing coverage. The following coverage ratio will be used:

$$\frac{\text{number of lines executed}}{\text{number of source code lines}}$$

and we will scale this ratio according to the size of the class.
The mark you will receive for the testing coverage task is determined using table 1:

| Coverage ratio range | % of testing mark |
|---|---|
| 0%-5% | 0% |
| 5%-20% | 20% |
| 20%-40% | 40% |
| 40%-60% | 60% |
| 60%-80% | 80% |
| 80%-100% | 100% |

Table 1: Mark assignment for testing

Note the top boundary for the Coverage ratio range is not inclusive except for 100%. Also, note that only the function stipulated in this specification will be considered to determine your mark. Remember that your main will be testing the instructor-provided code and as such it can only be assumed that the functions outlined in this specification are defined and implemented.

# 9 Upload checklist

The following files should be in the root of your archive

- Graph.java
- Edge.java
- Vertex.java
- Main.java
- Any textfiles needed by your Main

# 10 Allowed libraries

- java.nio.file.Files
- java.nio.file.Paths
- java.util.ArrayList
- java.util.HashSet
- java.util.Random
- java.util.IdentityHashMap
- java.util.Comparator
- java.util.Stack
- java.util.HashMap

# 11 Submission

You need to submit your source files on the FitchFork website (https://ff.cs.up.ac.za/). All methods need to be implemented (or at least stubbed) before submission. Place the above-mentioned files in a zip named uXXXXXXXX.zip where XXXXXXXX is your student number. Your code must be able to be compiled with the Java 8 standard.

For this practical, you will have 3 upload opportunities and your best mark will be your final mark. Upload your archive to the appropriate slot on the FitchFork website well before the deadline. **No late submissions will be accepted!**